## Experiment – 9 – RSA

## Program:

```
def gcd(a, b):
    while b: a, b = b, a % b
    return a

def modinv(a, m):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def is_prime(n):
    return all(n % i != 0 for i in range(2, int(n**0.5)+1))

p = int(input("Enter first prime (p): "))
q = int(input("Enter second prime (q): "))

if not (is_prime(p) and is_prime(q)):
    print("Both numbers must be prime!")
    exit()

n = p * q
phi = (p - 1) * (q - 1)
e = 65537 if gcd(65537, phi) == 1 else 3
d = modinv(e, phi)

print(f"Public Key: (e={e}, n={n})")
print(f"Private Key: (d={d}, n={n})")

msg = input("Enter message (only letters): ").upper()

cipher = [pow(ord(char), e, n) for char in msg]
print("Encrypted:", cipher)
decrypted = ''.join([chr(pow(c, d, n)) for c in cipher])
print("Decrypted:", decrypted)
```

## Output:

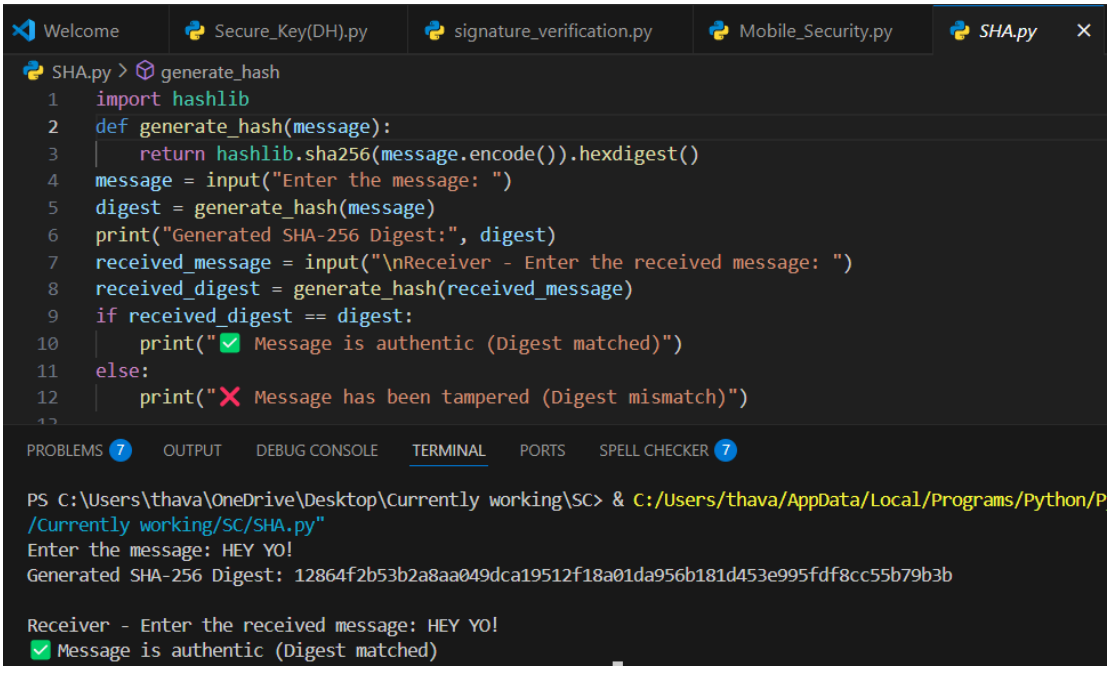

Welcome
Secure_Key(DH).py
signature_verification.py
Mobile_Security.py
RSA.py
RSA.py > ...
5 def modinv(a, m):
6 for x in range(1, m):
7 if (a * x) % m == 1:
8 return x
9 return None
10
11 def is_prime(n):
12 return all(n % i != 0 for i in range(2, int(n**0.5)+1))
13
14 p = int(input("Enter first prime (p): "))
15 q = int(input("Enter second prime (q): "))
16
17 if not (is_prime(p) and is_prime(q)):
18 print("Both numbers must be prime!")
19 exit()
PROBLEMS 9
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
SPELL CHECKER 9
PS C:\Users\thava\OneDrive\Desktop\Currently working\SC> & C:/Users/thava/AppData/Local/Programs/Python/P
/Currently working/SC/RSA.py"
Enter first prime (p): 53
Enter second prime (q): 61
Public Key: (e=65537, n=3233)
Private Key: (d=2753, n=3233)
Enter message (only letters): SCLAB
Encrypted: [2680, 641, 2726, 2790, 524]
Decrypted: SCLAB

## Experiment – 10 - Message Authentication using SHA algorithm

## Program:

```
import hashlib

def generate_hash(message):
    return hashlib.sha256(message.encode()).hexdigest()

message = input("Enter the message: ")

digest = generate_hash(message)

print("Generated SHA-256 Digest:", digest)

received_message = input("\nReceiver - Enter the received message: ")

received_digest = generate_hash(received_message)

if received_digest == digest:
    print("✅ Message is authentic (Digest matched)")

else:
    print("❌ Message has been tampered (Digest mismatch)")
```

## Output:

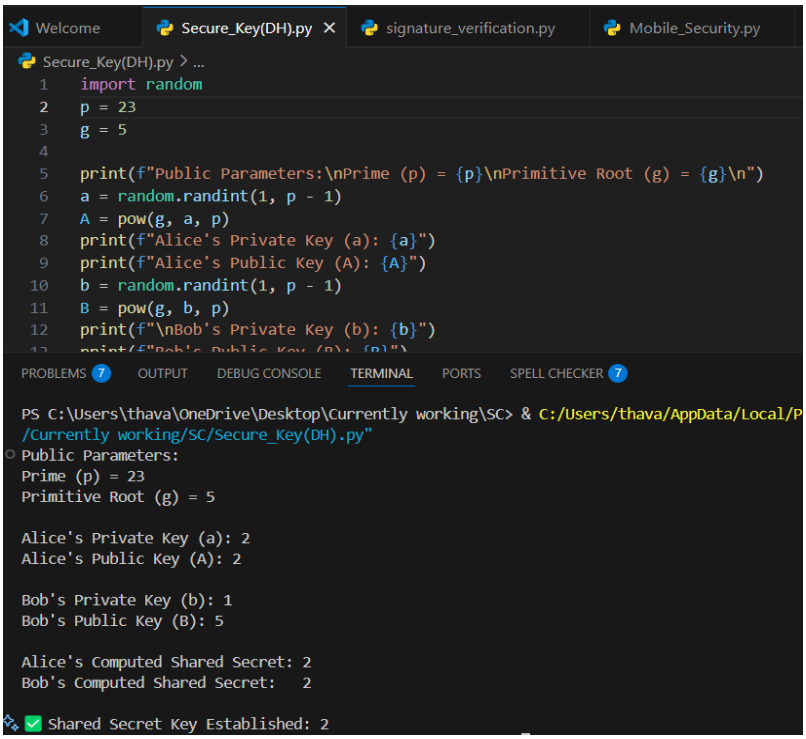


## Experiment – 11 - Secure key exchange

## Program:

```
import random
p = 23
g = 5
print(f"Public Parameters:\nPrime (p) = {p}\nPrimitive Root (g) = {g}\n")
a = random.randint(1, p - 1)
A = pow(g, a, p)
print(f"Alice's Private Key (a): {a}")
print(f"Alice's Public Key (A): {A}")
b = random.randint(1, p - 1)
B = pow(g, b, p)
print(f"\nBob's Private Key (b): {b}")
print(f"Bob's Public Key (B): {B}")
shared_secret_alice = pow(B, a, p)
shared_secret_bob = pow(A, b, p)

print(f"\nAlice's Computed Shared Secret: {shared_secret_alice}")
print(f"Bob's Computed Shared Secret:   {shared_secret_bob}")
if shared_secret_alice == shared_secret_bob:
    print(f"\n✅ Shared Secret Key Established: {shared_secret_alice}")
else:
    print("\n❌ Shared secrets don't match! Something went wrong.")
```

## Output:

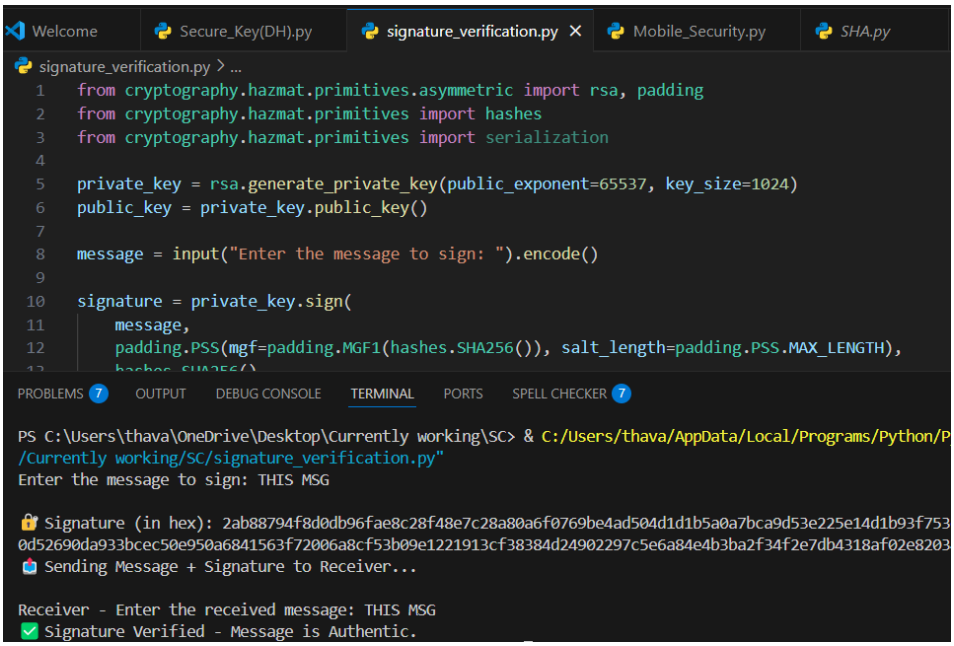


## Experiment – 12 - Digital Signature Generation

## Program:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives import serialization

private_key = rsa.generate_private_key(public_exponent=65537, key_size=1024)
public_key = private_key.public_key()
message = input("Enter the message to sign: ").encode()
signature = private_key.sign(
    message,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)
print("\n🔐 Signature (in hex):", signature.hex())
print("📤 Sending Message + Signature to Receiver...\n")
received_message = input("Receiver - Enter the received message: ").encode()
try:
    public_key.verify(signature, received_message,
padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
hashes.SHA256())

    print("✅ Signature Verified - Message is Authentic.")
except Exception as e:
    print("❌ Signature Invalid - Message Tampered or Not from Sender.")
```

## Output:



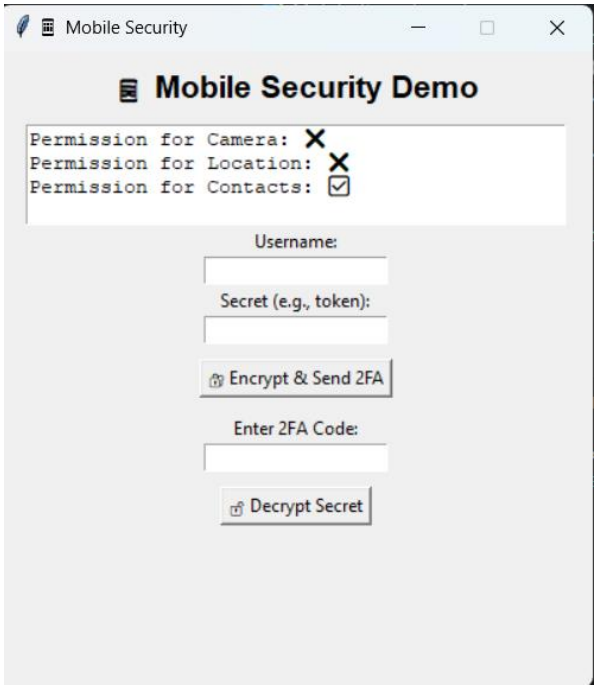
## Experiment – 13 – Mobile Security

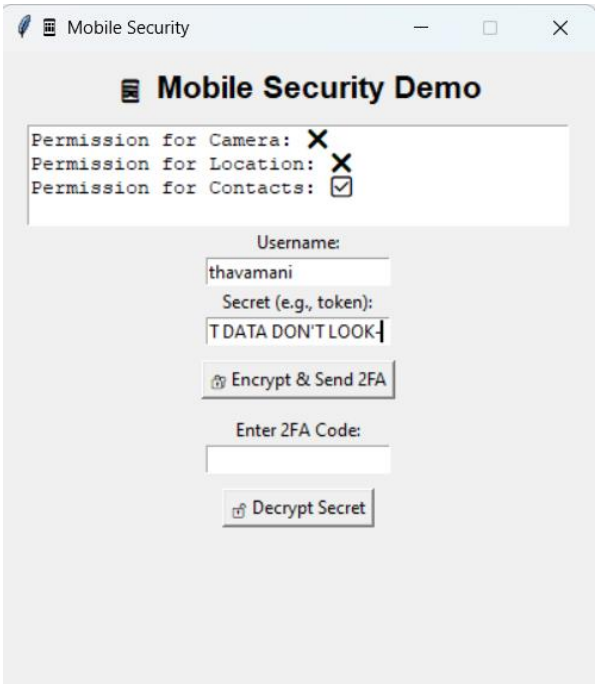
## Program:

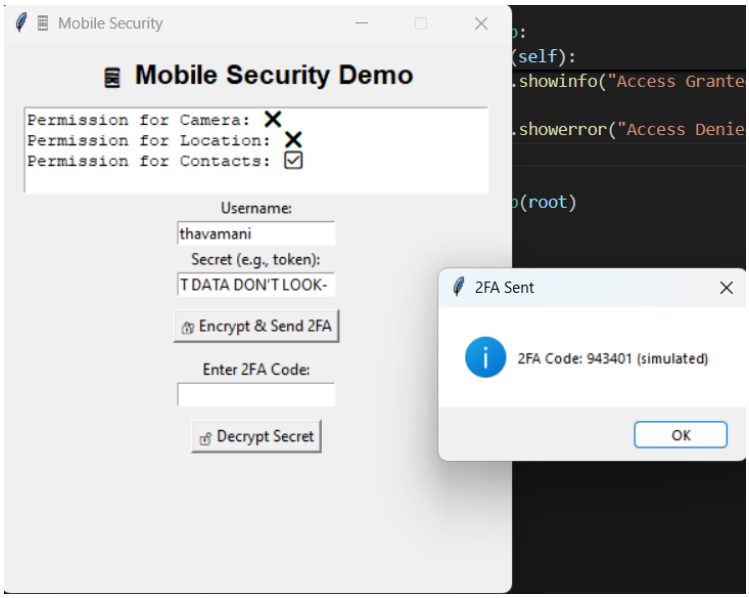
```
import tkinter as tk
from tkinter import messagebox
import base64
import hashlib
import random
from cryptography.fernet import Fernet

MASTER_PASSWORD = "10032005"

def generate_key(password):
    return base64.urlsafe_b64encode(hashlib.sha256(password.encode()).digest())

def encrypt_data(data, key):
    return Fernet(key).encrypt(data.encode())

def decrypt_data(token, key):
    return Fernet(key).decrypt(token).decode()

def generate_2fa():
    return str(random.randint(100000, 999999))

class MobileSecurityApp:
    def __init__(self, root):
        self.root = root
        self.root.title(" Mobile Security")
        self.root.geometry("400x430")
        self.root.resizable(False, False)

        self.key = generate_key(MASTER_PASSWORD)
        self.encrypted_data = None
        self.generated_2fa = None

        tk.Label(root, text=" Mobile Security Demo", font=("Arial", 16, "bold")).pack(pady=10)

        self.permission_text = tk.Text(root, height=4, width=45)
        self.permission_text.pack()
        self.display_permissions()

        tk.Label(root, text="Username:").pack()
        self.username_entry = tk.Entry(root)
        self.username_entry.pack()

        tk.Label(root, text="Secret (e.g., token):").pack()
```

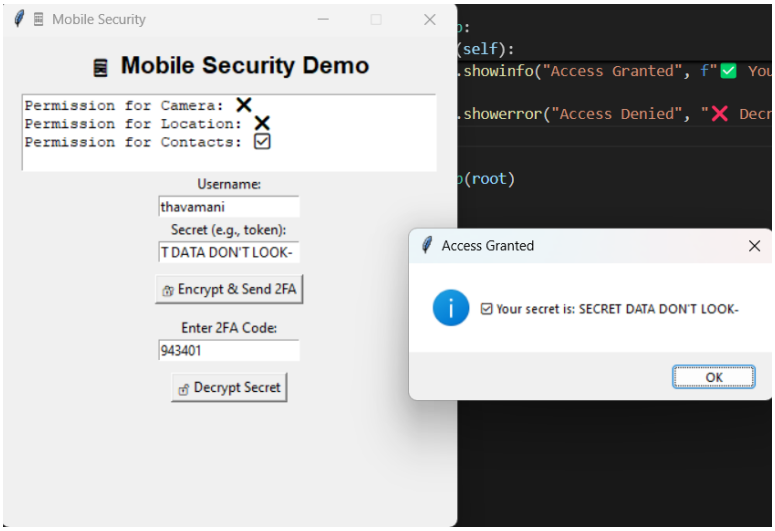
```
        self.secret_entry = tk.Entry(root)
        self.secret_entry.pack()

        tk.Button(root, text="🔐 Encrypt & Send 2FA",
command=self.encrypt_and_send_2fa).pack(pady=10)

        tk.Label(root, text="Enter 2FA Code:").pack()
        self.otp_entry = tk.Entry(root)
        self.otp_entry.pack()

        tk.Button(root, text="🔓 Decrypt Secret", command=self.decrypt_secret).pack(pady=10)

    def display_permissions(self):
        permissions = {"Camera": False, "Location": False, "Contacts": True}
        for perm, granted in permissions.items():
            status = "✅" if granted else "❌"
            self.permission_text.insert(tk.END, f"Permission for {perm}: {status}\n")
        self.permission_text.config(state='disabled')

    def encrypt_and_send_2fa(self):
        secret = self.secret_entry.get()
        if not secret:
            messagebox.showwarning("Input Error", "Please enter your secret.")
            return

        self.encrypted_data = encrypt_data(secret, self.key)
        self.generated_2fa = generate_2fa()
        messagebox.showinfo("2FA Sent", f"2FA Code: {self.generated_2fa} (simulated)")

    def decrypt_secret(self):
        if self.otp_entry.get() != self.generated_2fa:
            messagebox.showerror("2FA Failed", "❌ 2FA failed! Access denied.")
            return
        try:
            decrypted = decrypt_data(self.encrypted_data, self.key)
            messagebox.showinfo("Access Granted", f"✅ Your secret is: {decrypted}")
        except:
            messagebox.showerror("Access Denied", "❌ Decryption failed!")
root = tk.Tk()
app = MobileSecurityApp(root)
root.mainloop()
```

## Output:


Mobile Security
Mobile Security Demo
Permission for Camera: ✗
Permission for Location: ✗
Permission for Contacts: ☑
Username:
Secret (e.g., token):
Encrypt & Send 2FA
Enter 2FA Code:
Decrypt Secret


Mobile Security
Mobile Security Demo
Permission for Camera: ✗
Permission for Location: ✗
Permission for Contacts: ☑
Username:
thavamani
Secret (e.g., token):
T DATA DON'T LOOK-
Encrypt & Send 2FA
Enter 2FA Code:
Decrypt Secret


Mobile Security
Mobile Security Demo
Permission for Camera: ✗
Permission for Location: ✗
Permission for Contacts: ☑
Username:
thavamani
Secret (e.g., token):
T DATA DON'T LOOK-
Encrypt & Send 2FA
Enter 2FA Code:
Decrypt Secret
2FA Sent
2FA Code: 943401 (simulated)
OK


Mobile Security
Mobile Security Demo
Permission for Camera: ✗
Permission for Location: ✗
Permission for Contacts: ☑
Username:
thavamani
Secret (e.g., token):
T DATA DON'T LOOK-
Encrypt & Send 2FA
Enter 2FA Code:
943401
Decrypt Secret
Access Granted
Your secret is: SECRET DATA DON'T LOOK-
OK